Project Report: Caption Crafter AI

Submitted by Student Name: Abhijeet Wadnere

Reg No: 25BAI11429

Course: Fundamentals in AI and ML CSA2001

- **TITLE**

Caption Crafter AI: An Automated Instagram Caption Generator using Gemini 2.0 Flash

- **INTRODUCTION**

Caption Crafter AI is a lightweight web application designed to simplify social media content creation. Built with the Flask framework, it leverages Google's state-of-the-art Gemini 2.0 Flash generative model to transform user-provided image descriptions into engaging, mood-specific Instagram captions.

- **MOTIVATION OF THE PROJECT**

The primary motivation was to explore the integration of Large Language Models (LLMs) via REST APIs into a full-stack environment. By automating the creative process of writing captions, the project demonstrates how AI can solve everyday micro-tasks for content creators while maintaining a seamless user experience.

- **PROBLEM STATEMENT**

Content creators often face "writer's block" when trying to pair their visuals with appropriate text. Manual caption writing can be time-consuming, and finding the right "tone" (e.g., sarcastic vs.

inspirational) is difficult. Existing tools are often bloated or require paid subscriptions for simple generative tasks.

• OBJECTIVE OF THE PROJECT

- To develop a functional web interface for caption generation.
- To integrate the Gemini 2.0 Flash model using a direct REST API approach.
- To implement robust error handling, including a mock fallback system for offline or API-limited scenarios.
- To ensure the application is deployment-ready for cloud platforms like Render.

• EXISTING METHODS

- **Manual Writing:** Relying on human creativity.
- **Template-based Apps:** Using pre-written, static lists of captions.
- **High-end SaaS AI:** Comprehensive marketing tools (e.g., Jasper, Copy.ai) that are often behind paywalls.

• PROS AND CONS OF THE STATED METHODS

- **Pros:** Manual writing is authentic; SaaS tools provide high-quality long-form content.
- **Cons:** Manual writing is slow; template apps lack originality; SaaS tools are expensive and overly complex for a single Instagram post.

• HARDWARE AND SOFTWARE REQUIREMENTS

- **Hardware:** A standard PC/Laptop with internet connectivity for API communication.
- **Software:** * Operating System: Windows/Linux/macOS.

- Language: Python 3.11.7.
- Framework: Flask 3.0.0.
- API: Google Gemini API (gemini-2.0-flash).
- Libraries: requests, python-dotenv, gunicorn.

• METHODOLOGY AND GOAL

The project follows a **Modular Web-Service Architecture**. The goal is to capture user intent via a form, process that intent through a prompt-engineering layer, and fetch a response from a remote AI model.

• FUNCTIONAL MODULES DESIGN AND ANALYSIS

1. **Frontend Module:** A responsive HTML/CSS interface using Jinja2 templates for dynamic data rendering.
2. **Request Processor:** Handles form data and constructs the AI prompt.
3. **API Integration Module:** Manages HTTP POST requests to Google's Generative Language endpoint.
4. **Fallback/Safety Module:** Monitors for rate limits and provides mock data if the API is unavailable.

• ALGORITHM DEVELOPMENT

1. **Input:** User enters description and mood.
2. **Preprocessing:** Construct prompt: "Generate ONE short Instagram caption for: {description}. Mood: {mood}.".
3. **Throttling:** Check time.time() - last_request_time. If < 1.5s, wait to avoid 429 errors.
4. **API Call:** Send JSON payload to Gemini REST endpoint.

5. **Parsing:** Extract the text from candidates[0]['content']['parts'][0]['text'].
6. **Cleanup:** Remove markdown artifacts like asterisks or quotes.

• SOFTWARE ARCHITECTURAL DIAGRAM

The system follows a Client-Server-API pattern:

- **User Interface** (Browser) $\leftrightarrow$ **Flask Backend** (Python) $\leftrightarrow$ **Gemini API** (External Cloud).
- **Local Storage:** Environment variables (.env) for API keys.

• CODING

The core logic resides in app.py, using the requests library to communicate with the model:

Python

response = requests.post(url, json=payload, timeout=30) #

• OUTPUT

The output is a single, cleaned string displayed in a stylized result box on the web page. If the input is "Sunset" and mood is "Romantic," the output might be: *"Chasing sunsets and dreams with you. 🌅"*.

• KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

- **Prompt Engineering:** Optimized system instructions to ensure the model only returns the caption text.
- **Rate Limiting:** A REQUEST_DELAY of 1.5 seconds to protect the API quota.
- **Mock Fallback:** A generate_mock_caption function that triggers if GEMINI_API_KEY is missing.

• SIGNIFICANT PROJECT OUTCOMES

- Successfully achieved real-time AI content generation.
- Implemented a production-ready environment using gunicorn and Procfile.
- Created a clean UI with CSS animations and gradients for a modern look.

• TESTING AND REFINEMENT

Testing involved submitting empty strings, very long descriptions, and rapid-fire requests. This led to the refinement of the parsing logic to filter out "introductory text" (like "Here is your caption:") and the addition of time.sleep to prevent rate-limit crashes.

• PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

This tool can be integrated into social media management dashboards, used by small business owners to maintain their online presence, or scaled into a browser extension for direct use on Instagram.com.

• CONTRIBUTION/FINDINGS OF THE PROJECT

The project demonstrates that high-performance AI (Gemini 2.0 Flash) can be utilized without heavy SDKs, keeping the application's footprint extremely small. It also highlights the importance of "graceful degradation" through the use of fallback modes.

• LIMITATION/CONSTRAINTS OF THE SYSTEM

- **Render Free Tier:** Initial boot-up takes 40–60 seconds after inactivity.
- **API Quota:** Subject to Google’s free-tier rate limits.
- **Single-turn:** The app does not support a conversational "chat" history for refining captions.

• CONCLUSIONS

Caption Crafter AI successfully meets the objective of providing a fast, lightweight, and reliable way to generate social media content. The integration of AI via REST endpoints proves to be a robust method for building modern web applications.

• FUTURE ENHANCEMENTS

- **Image-to-Caption:** Allowing users to upload images directly using Gemini's multimodal capabilities.
- **History/Caching:** Storing previously generated captions in a local database.
- **Multi-Platform Support:** Adding specific formatting for LinkedIn or Twitter (X).